

CSCE 735 PARALLEL COMPUTING · MODULE 11 · LECTURE NOTES

Parallel Algorithms

The primitives, and the one that surprises everyone.

MODULE OUTCOMES

- Analyse an algorithm by work and depth.
- Implement parallel reduction and scan.
- Explain why scan is parallelisable despite appearing sequential.
- Implement parallel sorting.
- Explain the limits of the PRAM model.

1 Work and depth

Measure	Definition	Interpretation
Work, W	The total number of operations performed, summed over all processors.	What the computation costs in total. Should be close to the best serial complexity , or the algorithm is wasting effort.
Depth (span), D	The length of the longest chain of dependent operations.	The running time on infinitely many processors — the fundamental limit on how much parallelism the algorithm admits.
W/D	The ratio.	The available parallelism. If W/D is 1000, there is no point using more than about 1000 processors.

$$T_p \leq W/p + D \quad (\text{Brent's theorem})$$

The running time on p processors is bounded by the work divided among them plus the depth. **The two terms correspond to the two ways an algorithm can fail to scale:** too much work, or too long a critical path.

WORK-EFFICIENCY MATTERS AS MUCH AS DEPTH

An algorithm with $O(\log n)$ depth and $O(n \log n)$ work performs more total operations than the serial $O(n)$ algorithm it replaces. It is not work-efficient.

So at modest processor counts it loses to the serial version, and the crossover point — where the parallelism outweighs the extra work — may lie beyond any machine you have access to.

A work-efficient parallel algorithm performs $O(n)$ work, asymptotically matching the serial version, and accepts somewhat greater depth in exchange.

The two scan formulations in §2 illustrate this exactly. Hillis–Steele is simpler and does $O(n \log n)$ work at $O(\log n)$ depth; Blelloch does $O(n)$ work at twice the depth. **Blelloch usually wins in practice,** which is the opposite of what a depth-only analysis suggests, and it is why both measures are needed.

2 Reduce and scan

REDUCTION IS AN OBVIOUS TREE

Combine adjacent pairs, then pairs of those results, and so on. $O(n)$ work and $O(\log n)$ depth, which is optimal on both counts.

It requires only that the operator be associative (Module 06 §3), with the floating-point caveat that follows from that.

In practice, implementations are two-level: each thread reduces a contiguous chunk serially, and then the per-thread results are combined by tree.

This structure is what every real implementation does, and for good reasons: the serial phase has perfect cache locality, requires no synchronisation, and uses no atomics, while the tree phase is over only p elements and is correspondingly short. The pure tree is the theoretical form; the hybrid is the useful one.

```

INPUT:  3  1  7  0  4  1  6  3
OUTPUT: 3  4 11 11 15 16 22 25      (inclusive prefix sums)

HILLIS-STEELE (simple;  $O(n \log n)$  work,  $O(\log n)$  depth):
  for d = 1, 2, 4, 8, ...:
    in parallel: x[i] += x[i-d]    for i >= d
  Each round DOUBLES the reach; after log n rounds every element
  has accumulated everything to its left.

BLELLOCH (work-efficient;  $O(n)$  work,  $2 \log n$  depth):
  UP-SWEEP:  build a reduction tree, storing partial sums
  DOWN-SWEEP: push prefixes back down the same tree

```

Every output depends on all previous inputs, which makes scan look like the definition of a sequential computation. **The doubling-reach intuition is what makes the parallel result believable:** after the first round each element holds the sum of itself and its neighbour; after the second, of four elements; after the third, of eight. Logarithmically many rounds suffice.

SCAN IS THE MOST USEFUL PRIMITIVE IN PARALLEL COMPUTING

Stream compaction. Given a 0/1 flag per element, scan the flags — and each surviving element's exclusive prefix sum *is* its index in the compacted output. Filtering becomes a scan plus a scatter.

Radix sort. Each pass is a scan over digit-bucket flags to compute destination offsets, followed by a scatter. **The whole sort is a sequence of scans** (§3).

Variable-sized output allocation. When each thread produces a different number of results, scan the counts to give every thread its output offset — with no atomics and no contention.

And a great deal more: sparse matrix format construction, quicksort partitioning, run-length encoding and decoding, building acceleration structures (CSCE 647 Module 04), polynomial evaluation, and recurrence solving.

If a problem looks inherently sequential because of a running total, it is probably a scan. That recognition is one of the most valuable things in this course.

3 Sorting

Algorithm	Approach	Suits
Sample sort	Sample the data to choose splitters, bucket every element by splitter, then sort each bucket locally.	Distributed memory. One all-to-all communication and then independent local sorts — which maps perfectly onto MPI.
Radix sort	For each digit, scan the bucket flags to compute offsets and scatter.	GPUs. No comparisons, no data-dependent branching, hence no divergence (Module 08), and it is built entirely from scans and scatters. The dominant GPU sort.
Bitonic sort	A fixed network of compare-exchange operations, independent of the data.	Hardware and fixed-structure settings. $O(n \log^2 n)$ work — not work-efficient — and its complete regularity is sometimes worth that.
Parallel merge sort	Sort segments locally, then merge pairs in parallel.	Shared memory. The merge is the hard part — see below.
Parallel quicksort	Partition by scan, then recurse on both halves in parallel.	Good average case; sensitive to pivot choice, as serially.

MERGING IS HARDER TO PARALLELISE THAN SORTING

Merging two sorted sequences appears trivially sequential: compare the heads, emit the smaller, advance. Each decision depends on the previous one.

The parallel formulation uses a co-rank computation. For a given output position k , binary search to determine how many of the first k output elements come from sequence A and how many from B — which is determined by the data and can be found independently for any k .

Then each thread merges an independent segment, from its computed starting positions in both inputs, with no communication and no dependence on any other thread.

So the merge becomes $O(\log n)$ depth — the binary search — rather than $O(n)$. **This is the step that makes parallel merge sort work**, and it is a good example of a problem whose sequential appearance conceals an independent-subproblem structure, exactly as scan does.

4 Models and their limits

PRAM IS A USEFUL FICTION

The PRAM model posits p processors sharing a single memory, with uniform unit-cost access and perfect synchrony. Variants differ in whether concurrent reads and writes are permitted.

It is genuinely useful for reasoning about work and depth, for proving lower bounds, and for establishing that a problem is parallelisable at all — the scan result of §2 is a PRAM result.

And it omits essentially everything that determines practical performance: the memory hierarchy, finite bandwidth, cache coherence traffic, synchronisation cost, communication latency, and load imbalance. Every one of those has been a module of this course.

So a PRAM-optimal algorithm may be a poor practical choice, and frequently is. **Use work and depth as a guide, then measure** — which is Module 07's discipline applied at the level of algorithm selection rather than code tuning.

- **The external memory model** counts block transfers between a fast small level and a slow large one. It predicts cache and I/O behaviour that PRAM cannot see, and it is why B-trees (CSCE 608 Module 03) have the shape they do.
- **Cache-oblivious algorithms** achieve optimal transfer counts at every level of a hierarchy without knowing any of its parameters — typically by recursive subdivision. Elegant, and genuinely practical.
- **BSP (bulk synchronous parallel)** structures a computation as supersteps of local computation separated by communication and a barrier, and assigns costs to each. **Models distributed memory well** and underlies several graph processing systems.
- **LogP** models communication with four parameters — latency, overhead, gap, and processor count — capturing the distinction between network latency and the per-message CPU cost, which $\alpha\beta$ (Module 10) conflates.
- **And the roofline** (Module 07), which is the least theoretically ambitious and the most practically useful of all of them. **Each of these models exists because a simpler one mispredicted something important.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Blelloch — Prefix Sums and Their Applications (free)	The scan paper. The work-efficient algorithm and a long list of applications. Read it — it changes how you see sequential-looking problems.
Harris, Sengupta & Owens — Parallel Prefix Sum (Scan) with CUDA (free)	Both scan formulations implemented and measured on a GPU, including the bank conflict handling from Module 09.
Stanford CS149 — parallel algorithms lectures (free)	Work and depth analysis with worked examples.

Resource	What it covers for this module
JaJa — An Introduction to Parallel Algorithms	The PRAM-theoretic treatment. Library copy; useful for the lower bounds and the formal analysis.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Analyse three algorithms you know by work and depth, and compute the available parallelism of each.
- 2 Implement a tree reduction and a two-level reduction. Measure both and explain the difference.
- 3 Implement Hillis–Steele scan and count the operations. Confirm it is $O(n \log n)$.
- 4 Implement Blelloch scan and confirm it is $O(n)$. **Measure both at several sizes and thread counts and find where each wins.**
- 5 **Implement stream compaction using scan** and compare against a serial filter.
- 6 Implement variable-size output allocation with a scan, and compare against using an atomic counter.
- 7 Implement radix sort as a sequence of scans and scatters.
- 8 Implement parallel merge using co-rank and verify it against a serial merge.
- 9 Benchmark your sort against `std::sort` and against a library parallel sort. Report both.
- 10 Find a problem in your own code that looks sequential because of a running total, and reformulate it as a scan.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define work and depth, and state Brent's theorem.
- 2 What is available parallelism, and why does work-efficiency matter as much as depth?
- 3 Why is reduction a tree, and what do real implementations do instead?
- 4 Why does scan look sequential, and what is the doubling-reach intuition?
- 5 Compare Hillis–Steele and Blelloch on work and depth.
- 6 Name five applications of scan.
- 7 Name five parallel sorts and say what each suits.
- 8 Why is merging hard to parallelise, and what is the co-rank trick?
- 9 What does PRAM assume, what does it omit, and what follows?